

COMSATS UNIVERSITY ISLAMABAD

WAH CAMPUS

Department of Computer Science

DevOps Assignment # 01

Course Title: DevOps

Course Code: CSC323

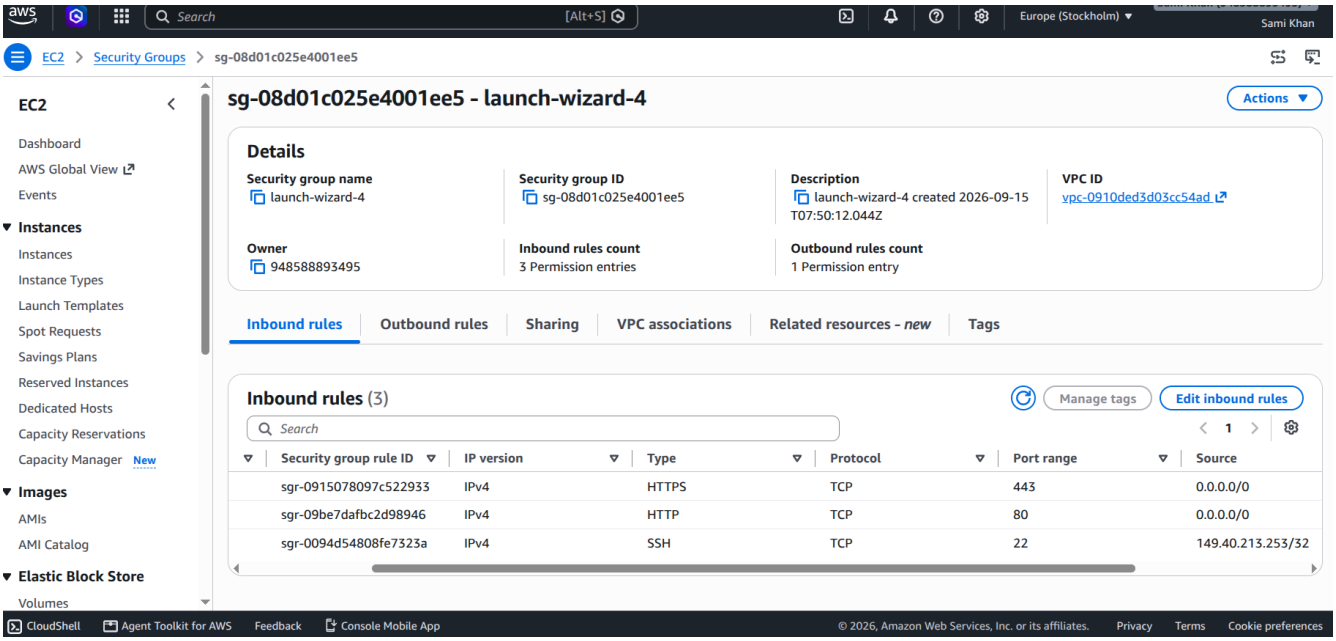
Instructor Name: Sir Ibrahim

Class: BSE-8A

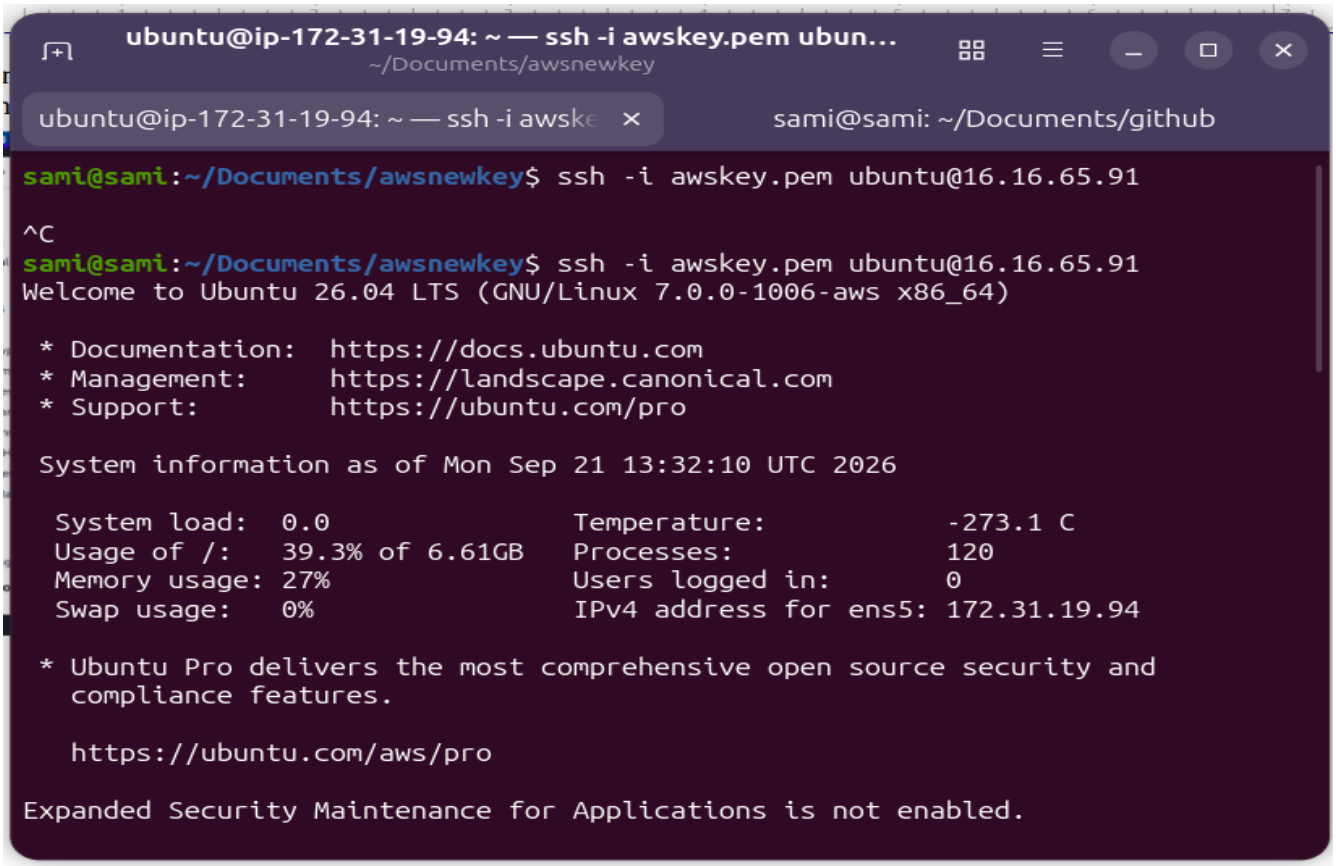
Name	Registration No.
Muhammad Sami	SP23-BSE-039

EC2 Docker Implementation:

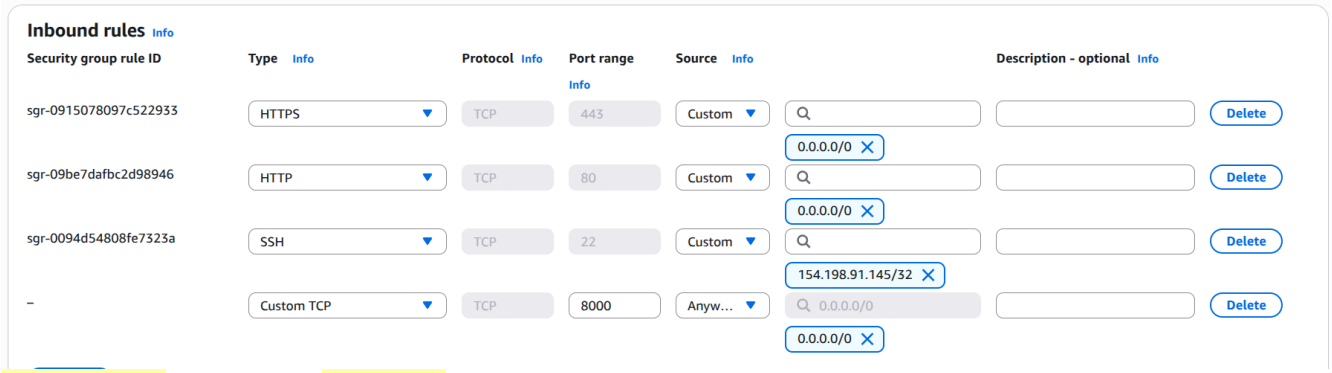
EC2 instance which was stopped now running and also connected to ubuntu Cloud VM in ubuntu terminal.Checking the EC2 security group in the AWS console before connecting. At this point only HTTPS (443), HTTP (80) and SSH (22) are allowed in, so port 8000 for Laravel still needs to be opened.



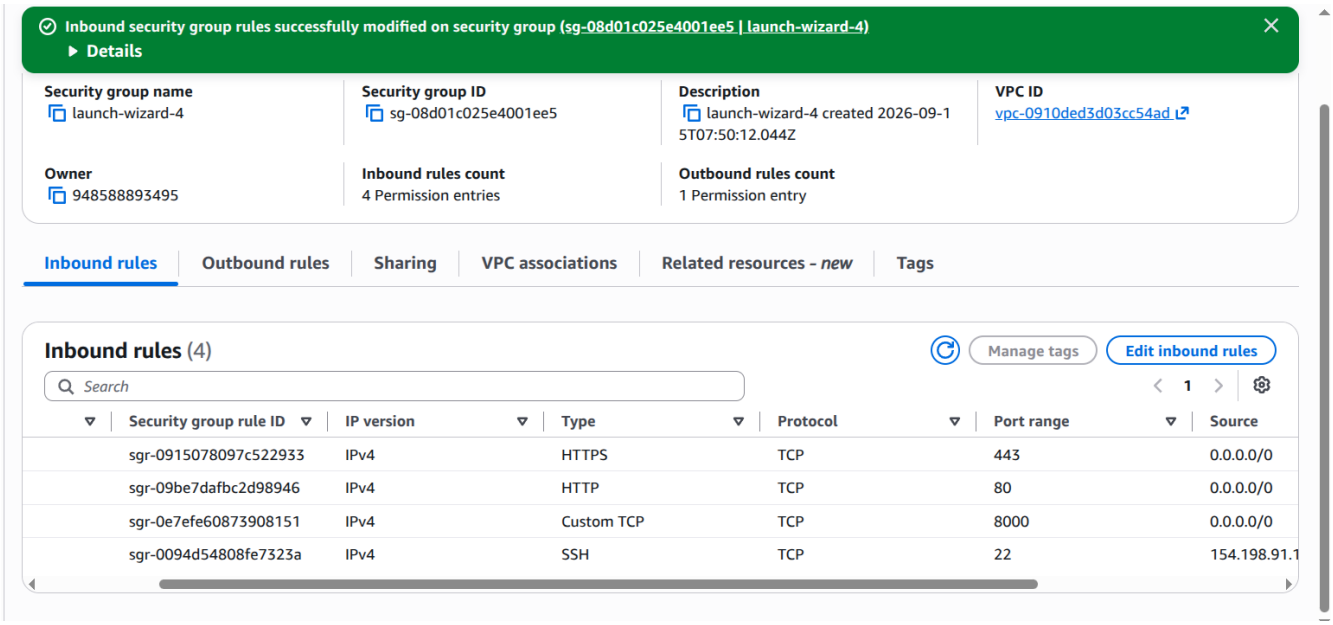
Connecting to the EC2 instance from the local terminal with ssh -i awskey.pem ubuntu@16.16.65.91. The Ubuntu 26.04 welcome banner confirms the SSH session succeeded and the instance is reachable.



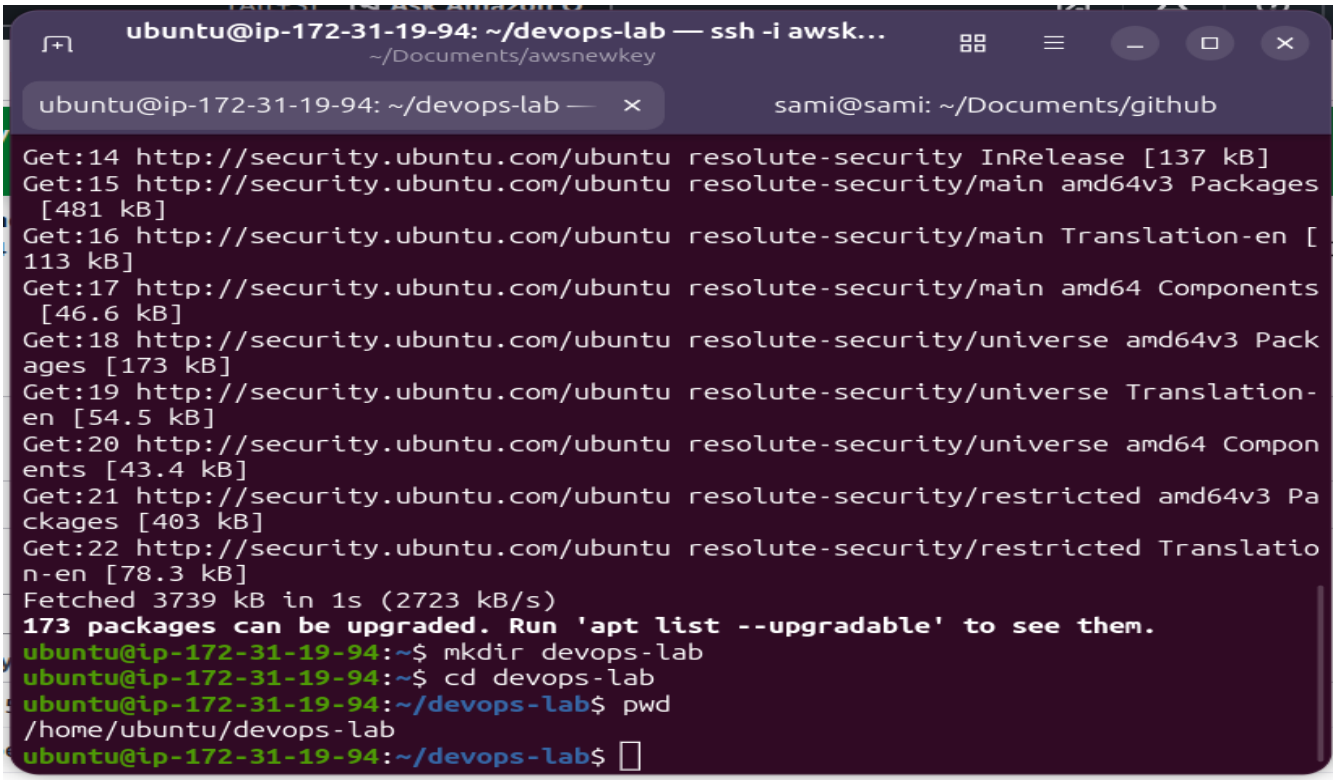
Editing the inbound rules of the security group to add a Custom TCP rule for port 8000. This is needed so the browser can reach the Laravel app once it starts listening on that port.



Custom TCP added with port 8000 Confirmation banner after saving the updated rules. The security group now shows 4 inbound rules, including the new Custom TCP rule on port 8000 open to 0.0.0.0/0..



Running sudo apt update to refresh the package lists, then creating and entering the working directory with mkdir devops-lab and cd devops-lab. pwd confirms the current path is /home/ubuntu/devops-lab.



Installing PHP and the supporting tools **Laravel needs:** sudo apt install -y php-cli php-mbstring php-xml php-curl php-zip unzip git. These provide the PHP CLI, the required PHP extensions, and Git.



Verifying that PHP and Git installed correctly with php --version and git --version. PHP 8.5.4 and Git 2.53.0 are confirmed, so the environment is ready for Composer.

```
Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-19-94:~/devops-lab$ php --version
PHP 8.5.4 (cli) (built: Sep  2 2026 14:42:18) (NTS)
Copyright (c) The PHP Group
Built by Ubuntu
Zend Engine v4.5.4, Copyright (c) Zend Technologies
    with Zend OPcache v8.5.4, Copyright (c), by Zend Technologies
ubuntu@ip-172-31-19-94:~/devops-lab$ git --version
git version 2.53.0
ubuntu@ip-172-31-19-94:~/devops-lab$
```

Installing Composer using the official installer script: downloading it with php -r "copy('https://getcomposer.org/installer', ...)" and running php composer-setup.php. The output confirms composer.phar was set up successfully.

```
ubuntu@ip-172-31-19-94:~/devops-lab$ php -r "copy('https://getcomposer.org/installer'
, 'composer-setup.php');"
ubuntu@ip-172-31-19-94:~/devops-lab$ php composer-setup.php
All settings correct for using Composer
Downloading...

Composer (version 2.10.3) successfully installed to: /home/ubuntu/devops-lab/composer
.phar
Use it: php composer.phar

ubuntu@ip-172-31-19-94:~/devops-lab$
```

Checking composer --version to confirm the install, then running composer create-project laravel/laravel laravel-app to scaffold a fresh Laravel project. Laravel 13.10.1 is downloaded and the project is created in the laravel-app folder.

```
ubuntu@ip-172-31-19-94:~/devops-lab$ composer --version
Composer version 2.10.3 2026-08-27 13:34:23
PHP version 8.5.4 (/usr/bin/php8.5)
Run the "diagnose" command to get more detailed diagnostics output.
ubuntu@ip-172-31-19-94:~/devops-lab$ composer create-project laravel/laravel laravel-
app
Creating a "laravel/laravel" project at "./laravel-app"
Installing laravel/laravel (v13.10.1)
  - Downloading laravel/laravel (v13.10.1)
  - Installing laravel/laravel (v13.10.1): Extracting archive
Created project in /home/ubuntu/devops-lab/laravel-app
> @php -r "file_exists('.env') || copy('.env.example', '.env');"
Loading composer repositories with package information
Updating dependencies
Lock file operations: 109 installs, 0 updates, 0 removals
  - Locking brick/math (0.19.1)
  - Locking carbonphp/carbon-doctrine-types (3.2.1)
  - Locking dflydev/dot-access-data (v3.0.3)
  - Locking doctrine/inflector (2.1.0)
```

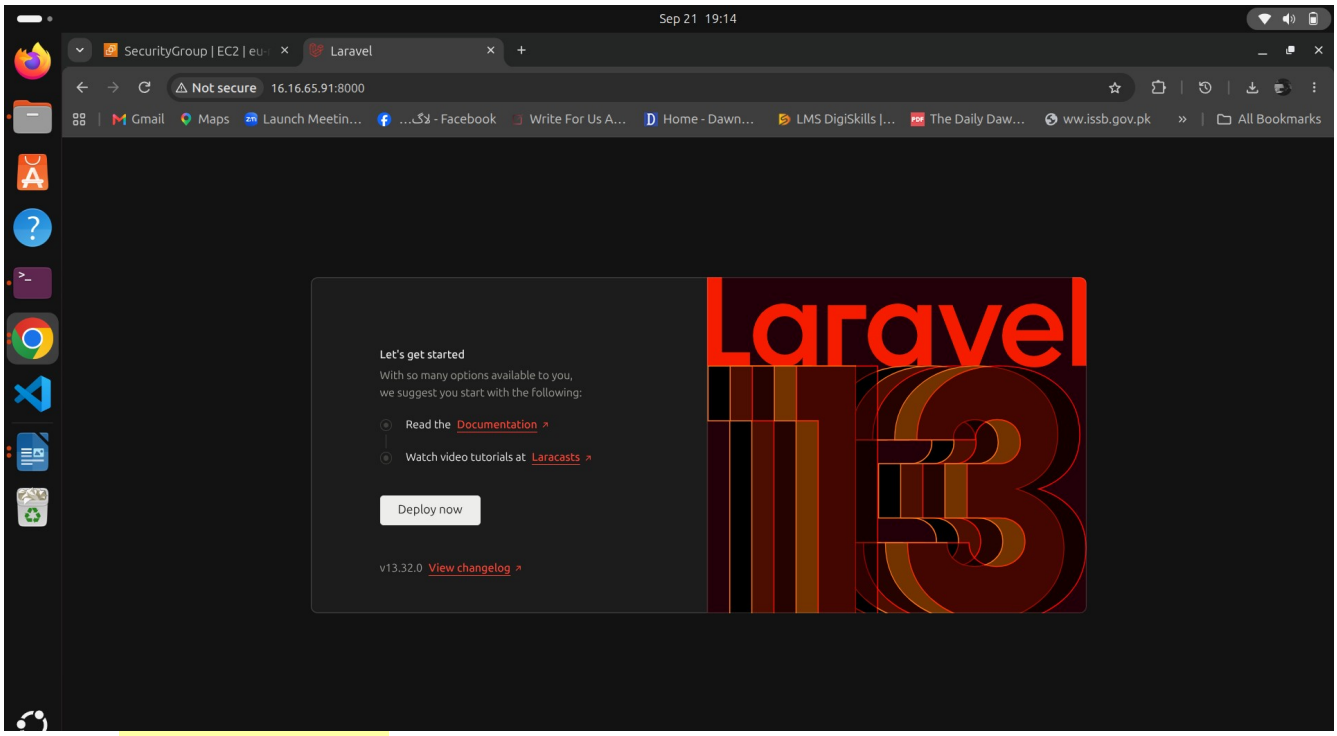
Moving into the project with cd laravel-app, listing its contents with ls, then starting the dev server with php artisan serve --host=0.0.0.0 --port=8000. The --host=0.0.0.0 flag lets the server accept connections from outside the VM, not just localhost.



.evn file configuration was changes and laravel welcome page shows without docker.The server is running on http://0.0.0.0:8000, and the request log shows an incoming GET request for /favicon.ico. This confirms the app is actually receiving and responding to HTTP traffic.



Opening the EC2 public IP on port 8000 in the browser and confirming the **Laravel welcome page loads.** This proves the app works directly on the VM before anything is containerized.



Checking docker --version first , Docker isn't installed yet, so the system lists a few install options. Installing it instead with sudo apt install -y docker.io, which pulls in the Docker engine and its dependencies.

```
^Cubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ docker --version
Command 'docker' not found, but can be installed with:
sudo snap install docker          # version 29.8.0, or
sudo snap install docker          # version 29.6.1
sudo apt  install podman-docker   # version 5.7.0+ds2-3build1
sudo apt  install docker.io       # version 29.1.3-0ubuntu4.1
See 'snap info <snapname>' for additional versions.
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo apt install -y docker.io
sudo systemctl enable --now docker
Installing:
  docker.io

Installing dependencies:
  bridge-utils  containerd  dns-root-data  dnsmasq-base  pigz  runc  ubuntu-fan

Suggested packages:
  ifupdown         | cgroup-lite  docker-compose-v2  rootlesskit
  aufs-tools       debootstrap    docker-doc         zfs-fuse
  cgroupfs-mount   docker-buildx  rinse              | zfsutils
```

Now we have installed the docker and tested it with hello-world, Running sudo docker run hello-world to sanity-check the Docker installation. The "Hello from Docker!" message confirms the daemon is reachable and can pull and run **images** correctly.

```
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

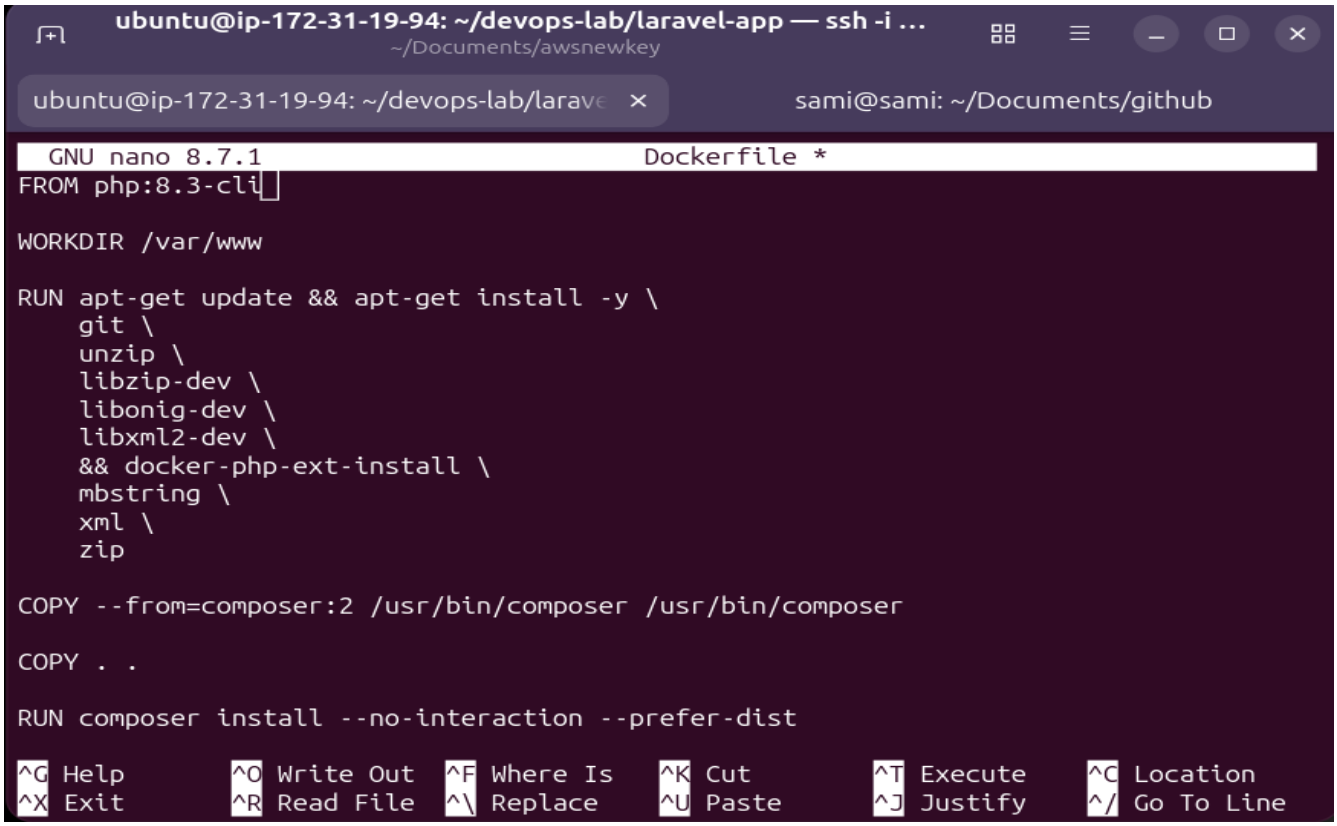
Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
```

Writing the Dockerfile with nano inside the project folder. It starts from php:8.3-cli, sets /var/www as the working directory, installs the needed system packages and PHP extensions, copies in Composer and the project files, then runs composer install.



```
ubuntu@ip-172-31-19-94: ~/devops-lab/laravel-app — ssh -i ...
~/Documents/awsnewkey
ubuntu@ip-172-31-19-94: ~/devops-lab/larave ×   sami@sami: ~/Documents/github

ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ nano Dockerfile
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ cat Dockerfile
FROM php:8.3-cli

WORKDIR /var/www

RUN apt-get update && apt-get install -y \
    git \
    unzip \
    libzip-dev \
    libonig-dev \
    libxml2-dev \
    && docker-php-ext-install \
    mbstring \
    xml \
    zip

COPY --from=composer:2 /usr/bin/composer /usr/bin/composer

COPY . .

RUN composer install --no-interaction --prefer-dist

EXPOSE 8000
```

Kicking off the image build with sudo docker build -t laravel-welcome . The build context is uploaded and Docker starts pulling and building the php:8.3-cli base image layer by layer.

```
ubuntu@ip-172-31-19-94: ~/devops-lab/laravel-app — ssh -i ...
~/Documents/awsnewkey
ubuntu@ip-172-31-19-94: ~/devops-lab/larave ×   sami@sami: ~/Documents/github

e/ubuntu/Dockerfile: no such file or directory
ubuntu@ip-172-31-19-94:~$ cd ~/devops-lab/laravel-app
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker build -t laravel-welcome
 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  56.72MB
Step 1/8 : FROM php:8.3-cli
8.3-cli: Pulling from library/php
ea3ed2b447c3: Pulling fs layer
67ce88467cba: Pulling fs layer
6b37362b3da7: Pulling fs layer
41695bc734f3: Pulling fs layer
401194ef010f: Pulling fs layer
073620ce3b4f: Pulling fs layer
fb27eec27d69: Pulling fs layer
9324a3e994b3: Pulling fs layer
936fc8bd0ccb: Pulling fs layer
e4666e6401d4: Pulling fs layer
e4666e6401d4: Download complete
ea3ed2b447c3: Download complete
41695bc734f3: Download complete
```

The build finishes successfully Docker reports **"Successfully built"** and **"Successfully tagged laravel-welcome:latest"**, so the image is ready to run as a container.

```
 ---> Removed intermediate container c0b2348ea421
 ---> 227d4f0131d9
Step 7/8 : EXPOSE 8000
 ---> Running in 1bd5532f6339
 ---> Removed intermediate container 1bd5532f6339
 ---> 44dec376eb6a
Step 8/8 : CMD ["php", "artisan", "serve", "--host=0.0.0.0", "--port=8000"]
 ---> Running in fce8cf55cfd6
 ---> Removed intermediate container fce8cf55cfd6
 ---> 8b23782521af
Successfully built 8b23782521af
Successfully tagged laravel-welcome:latest
```

Checking sudo docker images to confirm laravel-welcome now exists alongside the composer, hello-world and php base images, then starting the container with sudo docker run -d --name laravel-app -p 8000:8000 laravel-welcome. docker ps shows the container up with the port mapped.

```
Successfully built 8b23782521af
Successfully tagged laravel-welcome:latest
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker images
                                                 i Info →  U  In Use
IMAGE                    ID             DISK USAGE   CONTENT SIZE   EXTRA
composer:2               a5f59b9fd2fa       315MB         79.9MB
hello-world:latest       5e2309035332      25.9kB         9.49kB    U
laravel-welcome:latest   8b23782521af       968MB          240MB
php:8.3-cli              83024befab01       750MB          187MB
php:8.4-cli              2b87a5d0c046       762MB          192MB
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker run -d --name laravel-ap
p -p 8000:8000 laravel-welcome
d372077ef1fbc4dcfe4c807bb390c8336e7f4b68a67f6c0f4fd479cd43c4b0c8
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED         STATUS
    PORTS                                       NAMES
d372077ef1fb   laravel-welcome   "docker-php-entrypoi…"   9 seconds ago   Up 8 second
s   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp   laravel-app
```

Testing the Laravel container,**Laravel's server running**.Then openning browser on chrome to test

http://51.20.92.190:8000/.

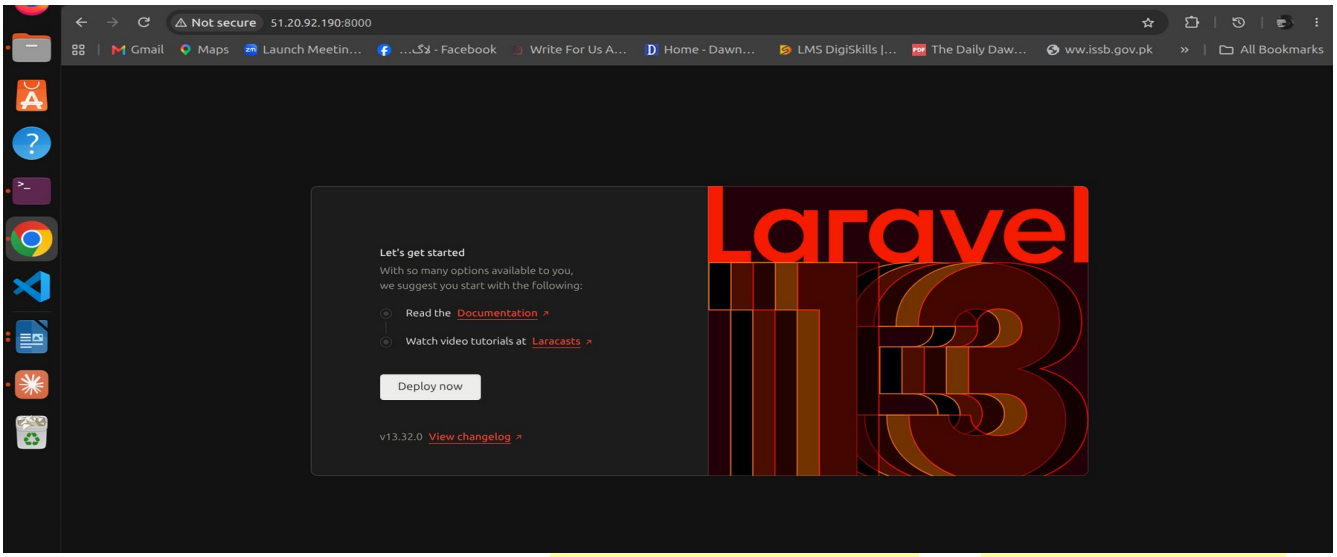
A follow-up check of docker images and docker ps, confirming the laravel-welcome image and the running laravel-app container are still healthy before testing it in the browser.

```
IMAGE                     ID             DISK USAGE   CONTENT SIZE   EXTRA
composer:2                a5f59b9fd2fa       315MB         79.9MB
hello-world:latest        5e2309035332      25.9kB         9.49kB    U
laravel-welcome:latest    8b23782521af       968MB          240MB
php:8.3-cli               83024befab01       750MB          187MB
php:8.4-cli               2b87a5d0c046       762MB          192MB
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker run -d --name laravel-ap
p -p 8000:8000 laravel-welcome
d372077ef1fbc4dcfe4c807bb390c8336e7f4b68a67f6c0f4fd479cd43c4b0c8
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED         STATUS
    PORTS                                       NAMES
d372077ef1fb   laravel-welcome   "docker-php-entrypoi…"   9 seconds ago   Up 8 second
s   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp   laravel-app
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker logs laravel-app

   INFO  Server running on [http://0.0.0.0:8000].

  Press Ctrl+C to stop the server
```

Now testing in browser http://51.20.92.190:8000/,Opening **http://51.20.92.190:8000/** in the browser to test the containerized app. The Laravel welcome page loads correctly, proving the container is serving traffic on the mapped port.



Cleaning up the standalone container with sudo docker stop laravel-app and sudo docker rm laravel-app, then confirming with docker ps that nothing is left running. This clears the way for testing Docker Compose next.

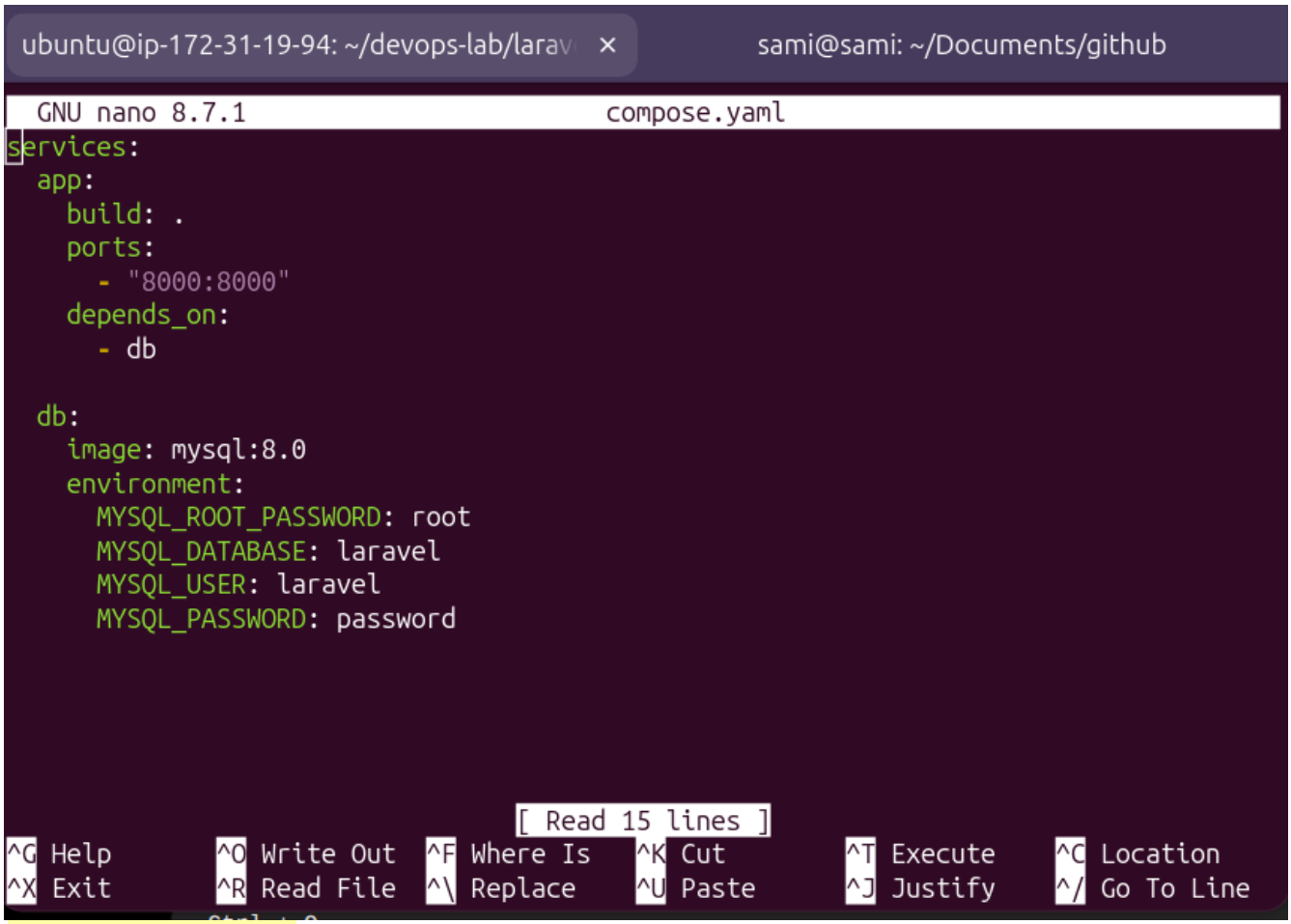
```
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker logs laravel-app

   INFO  Server running on [http://0.0.0.0:8000].

  Press Ctrl+C to stop the server

ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker stop laravel-app
laravel-app
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker rm laravel-app
laravel-app
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$
```

Creating **compose.yaml** with nano to define the multi-container stack. It declares an app service built from the local Dockerfile (mapped to port 8000) and a db service using the mysql:8.0 image, with the root password, database name, and app user/password set as environment variables.



docker compose up -d is being run in terminal this will:

1. Build the Laravel `app` service from your Dockerfile.
2. Download the MySQL 8.0 image.
3. Create the Laravel container.
4. Create the MySQL container.
5. Create a Docker Compose network connecting them.

Starting the stack with **sudo docker compose up -d.** Compose pulls the MySQL image and starts building the Laravel app image, then begins creating the network and containers for both services.

```
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker compose up -d
[+] Running 14/14
 ✔ db Pulled                                                          13.9s
   ✔ 96d30d9fbee8 Pull complete                                        0.5s
   ✔ 4c8a3e0d4e4b Pull complete                                        0.4s
   ✔ 6ef6c7b50a93 Pull complete                                        0.5s
   ✔ e3e5d1ac74c1 Pull complete                                        0.6s
   ✔ a63160a5eda1 Pull complete                                        5.4s
   ✔ 0d74d296605b Pull complete                                        4.1s
   ✔ 297d04cfe470 Pull complete                                        0.5s
   ✔ 7534d1db9f8d Pull complete                                        5.5s
   ✔ edf85873f64e Pull complete                                        3.8s
   ✔ ab24264a27e9 Pull complete                                        0.5s
   ✔ 49ec2dab01d9 Pull complete                                       12.3s
   ✔ 796812c73292 Download complete                                    0.0s
   ✔ af166387641d Download complete                                    0.5s
WARN[0014] Docker Compose is configured to build using Bake, but buildx isn't install
ed
[+] Building 74.4s (14/14) FINISHED                          docker:default
 => [app internal] load build definition from Dockerfile              0.1s
 => => transferring dockerfile: 460B                                  0.0s
 => [app internal] load metadata for docker.io/library/php:8.4-cli    0.0s
 => [app internal] load metadata for docker.io/library/composer:2     0.0s
 => [app internal] load .dockerignore                                 0.0s
```

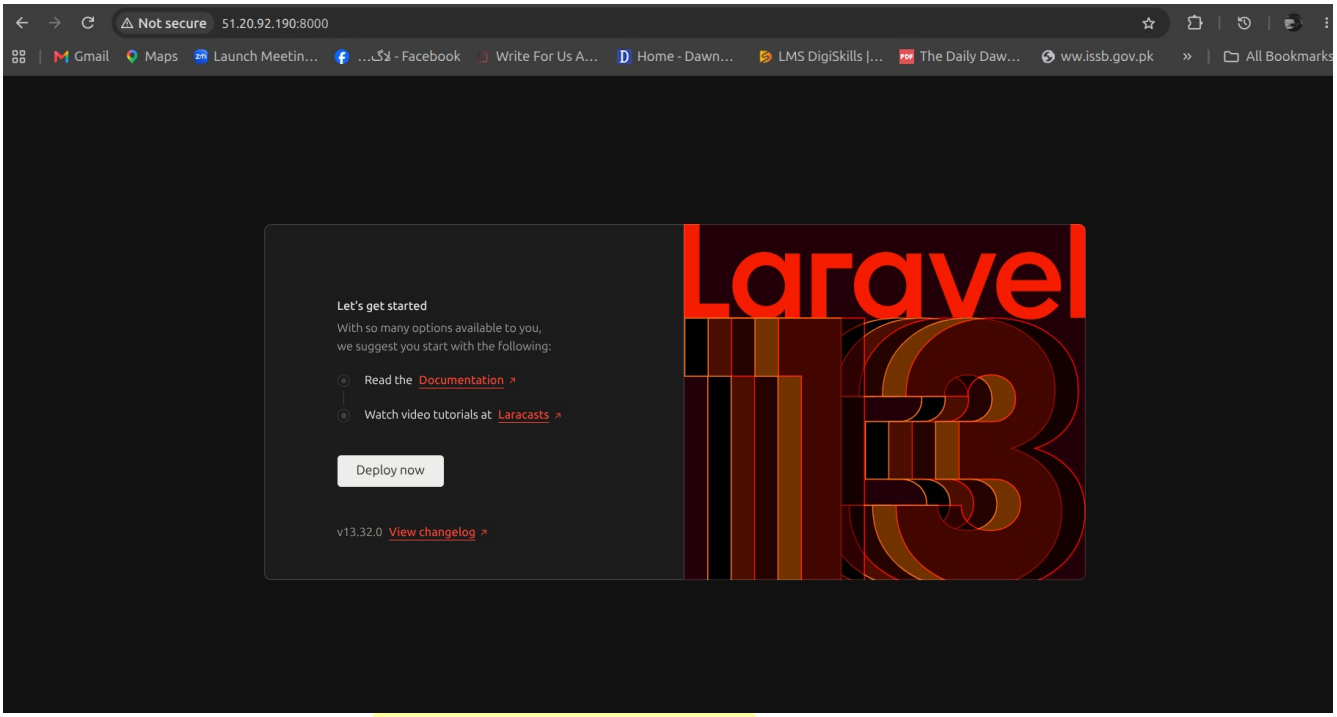
The build continues Compose finishes the app image layers, creates the laravel-app_default network, and reports both the db and app containers as Created and Started.

```
 => [app internal] load build context                                 4.9s
 => => transferring context: 50.08MB                                  4.7s
 => [app] FROM docker.io/library/composer:2@sha256:a5f59b9fd2faf31218632be4809  1.6s
 => => resolve docker.io/library/composer:2@sha256:a5f59b9fd2faf31218632be4809  0.0s
 => [app stage-0 2/6] WORKDIR /var/www                                0.3s
 => [app stage-0 3/6] RUN apt-get update && apt-get install -y     git     un  49.5s
 => [app stage-0 4/6] COPY --from=composer:2 /usr/bin/composer /usr/bin/compos  0.2s
 => [app stage-0 5/6] COPY . .                                        7.4s
 => [app stage-0 6/6] RUN composer install --no-interaction --prefer-dist     3.1s
 => [app] exporting to image                                         12.3s
 => => exporting layers                                               9.6s
 => => exporting manifest sha256:b6727cc19504313ea04e1382059197c781805ff742a8a  0.0s
 => => exporting config sha256:d589360d4c668eb80c514143d9f3810c1e7ce8007268340  0.0s
 => => exporting attestation manifest sha256:63443a8049af67b1677b14c85d305ba92  0.0s
 => => exporting manifest list sha256:0847cdd402ca072dc5d7ad053b45f6345cba4674  0.0s
 => => naming to docker.io/library/laravel-app-app:latest             0.0s
 => => unpacking to docker.io/library/laravel-app-app:latest          2.6s
 => [app] resolving provenance for metadata file                      0.1s
[+] Running 4/4
 ✔ app                          Built                                0.0s
 ✔ Network laravel-app_default  Created                              0.1s
 ✔ Container laravel-app-db-1   Started                              0.5s
 ✔ Container laravel-app-app-1  Started                              0.5s
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$
```

Running sudo docker compose ps to confirm both containers are up: laravel-app-app-1 on port 8000 and laravel-app-db-1 for MySQL. This shows the two-container stack being managed as a single application, then opening the app in the browser again to re-confirm it.

```
 => [app stage-0 5/6] COPY . .                                              7.4s
 => [app stage-0 6/6] RUN composer install --no-interaction --prefer-dist   3.1s
 => [app] exporting to image                                               12.3s
 => => exporting layers                                                     9.6s
 => => exporting manifest sha256:b6727cc19504313ea04e1382059197c781805ff742a8a  0.0s
 => => exporting config sha256:d589360d4c668eb80c514143d9f3810c1e7ce8007268340  0.0s
 => => exporting attestation manifest sha256:63443a8049af67b1677b14c85d305ba92  0.0s
 => => exporting manifest list sha256:0847cdd402ca072dc5d7ad053b45f6345cba4674  0.0s
 => => naming to docker.io/library/laravel-app-app:latest                   0.0s
 => => unpacking to docker.io/library/laravel-app-app:latest                2.6s
 => [app] resolving provenance for metadata file                            0.1s
[+] Running 4/4
 ✔ app                          Built                                       0.0s
 ✔ Network laravel-app_default  Created                                     0.1s
 ✔ Container laravel-app-db-1   Started                                     0.5s
 ✔ Container laravel-app-app-1  Started                                     0.5s
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker compose ps
NAME                 IMAGE              COMMAND                  SERVICE   CREATED
    STATUS          PORTS
laravel-app-app-1    laravel-app-app    "docker-php-entrypoi…"   app       2 minutes ag
o   Up 2 minutes    0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
laravel-app-db-1     mysql:8.0          "docker-entrypoint.s…"   db        2 minutes ag
o   Up 2 minutes    3306/tcp, 33060/tcp
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$
```

The **Laravel welcome page** still loads fine while the Compose stack is running, confirming the app container behaves the same way under Compose as it did standalone.



Tearing the stack down with **sudo docker compose down.** Both containers and the network are removed cleanly, and docker ps confirms nothing is left running.

```
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker compose down
[+] Running 3/3
 ✔ Container laravel-app-app-1  Removed                                  11.4s
 ✔ Container laravel-app-db-1   Removed                                   1.5s
 ✔ Network laravel-app_default  Removed                                   0.1s
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$
```

Logging in to Docker Hub with sudo docker login using the web-based device login flow entering the one-time code in the browser and confirming. The terminal reports **"Login Succeeded".**

```
ubuntu@ip-172-31-19-94: ~/devops-lab/larav ×        sami@sami: ~/Documents/github

r.com/activate

Waiting for authentication in the browser…
^Clogin canceled
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker login

USING WEB-BASED LOGIN

i Info → To sign in with credentials on the command line, use 'docker login -u <usern
ame>'


Your one-time device confirmation code is: BDTN-HTNG
Press ENTER to open your browser or submit your device code here: https://login.docke
r.com/activate

Waiting for authentication in the browser…

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$
```

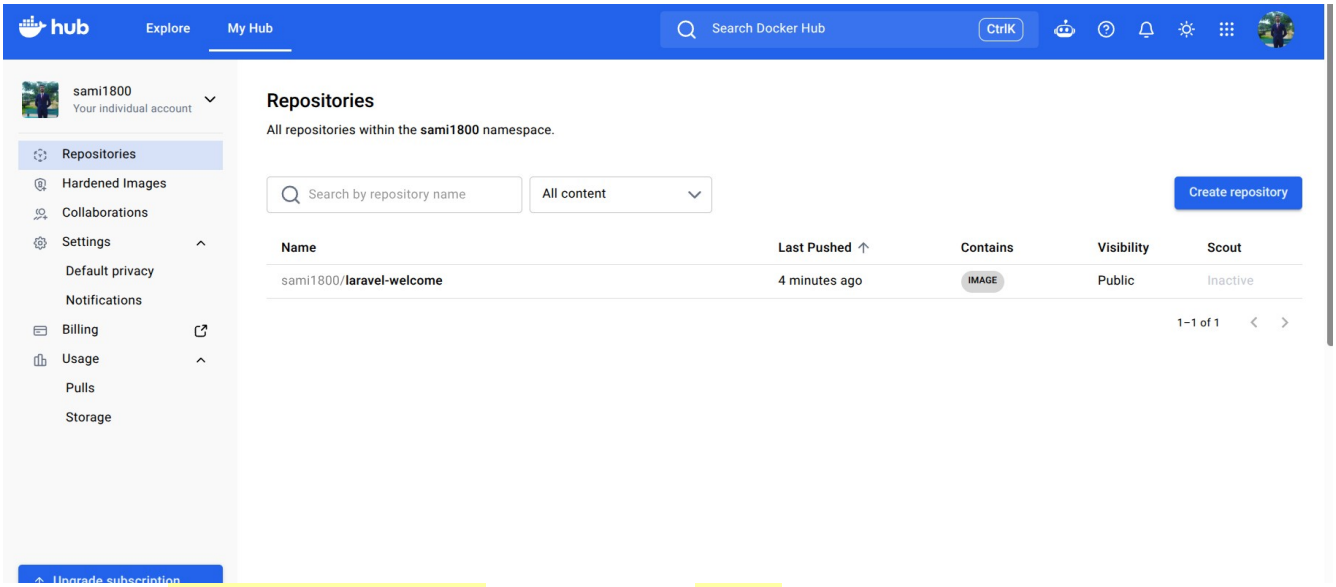
Tagging the local image for **Docker Hub** with **sudo docker tag laravel-app-app:latest sami1800/laravel-welcome:1.0, t**hen starting the push with sudo docker push. Each image layer is uploaded one at a time.

```
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker tag laravel-app-app:late
st sami1800/laravel-welcome:1.0
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker images
                                                  i Info →  U  In Use
IMAGE                          ID             DISK USAGE   CONTENT SIZE   EXTRA
laravel-app-app:latest         cad108151c90       968MB          240MB
mysql:8.0                      7dcddc01f13b       1.1GB          249MB
sami1800/laravel-welcome:1.0   cad108151c90       968MB          240MB
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker push sami1800/laravel-we
lcome:1.0
The push refers to repository [docker.io/sami1800/laravel-welcome]
99091603b490: Pushed
96fc9acdcf33: Pushed
6b37362b3da7: Pushed
cb3bed725cd6: Pushed
aefd7737bbe4: Pushed
d6aed5f5cedf: Pushed
81055ac1a39c: Pushed
0682a2b74e5f: Pushed
ea9b1dd160ca: Pushed
1303a0d4ee60: Pushed
4f4fb700ef54: Pushed
4383341a07a2: Pushed
d8f4c35935ec: Pushed
```

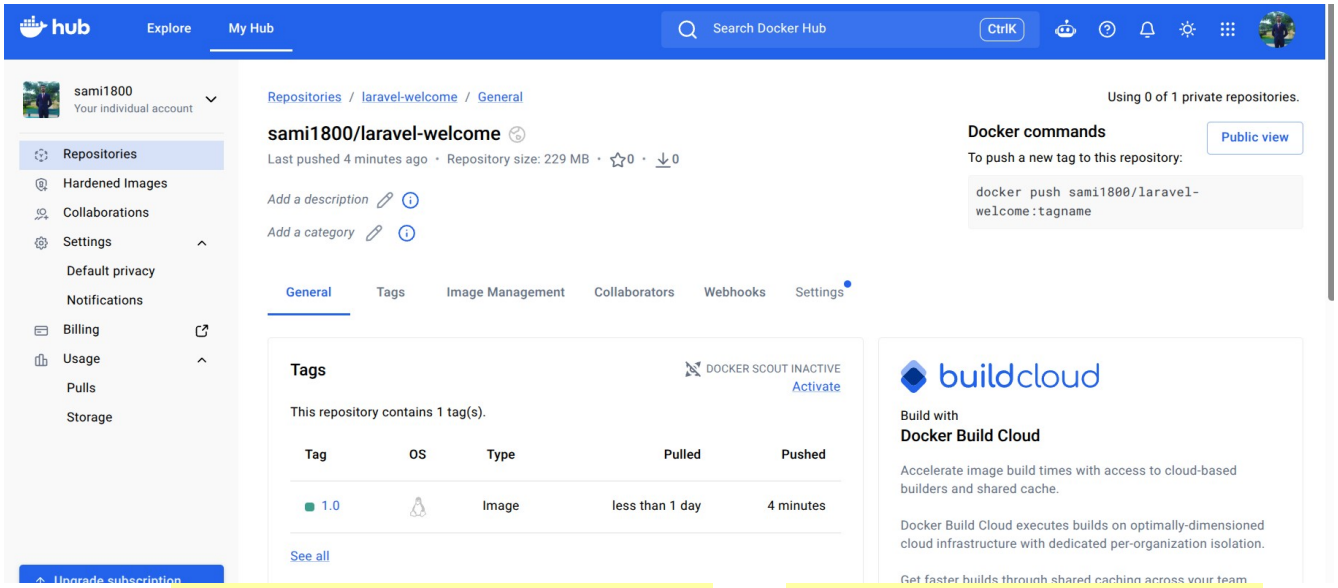
The push finishes every layer shows as Pushed and Docker reports the final digest, confirming sami1800/laravel-welcome:1.0 is now available on Docker Hub.

```
The push refers to repository [docker.io/sami1800/laravel-welcome]
99091603b490: Pushed
96fc9acdcf33: Pushed
6b37362b3da7: Pushed
cb3bed725cd6: Pushed
aefd7737bbe4: Pushed
d6aed5f5cedf: Pushed
81055ac1a39c: Pushed
0682a2b74e5f: Pushed
ea9b1dd160ca: Pushed
1303a0d4ee60: Pushed
4f4fb700ef54: Pushed
4383341a07a2: Pushed
d8f4c35935ec: Pushed
866e8ec4e67d: Pushed
9b9c9ac6d652: Pushed
f665ecff8fdb: Pushed
1.0: digest: sha256:cad108151c905d9f8aa41c58561c629314b6a958dde496ebd63f27f43191d794
size: 856
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$
```

Checking the **Docker Hub dashboard** to confirm the repository was created. sami1800/laravel-welcome shows up under My Hub, pushed a few minutes earlier and marked Public.



Opening the **repository's detail page**, which lists the 1.0 tag along with its size and pull command confirming the image is properly published and ready to be pulled from another machine.



Back on the EC2 instance, pulling the published image with **sudo docker pull sami1800/laravel-welcome:1.0.** The digest matches what was pushed, and Docker reports the image is up to date.

```
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker pull sami1800/laravel-welcome:1.0
1.0: Pulling from sami1800/laravel-welcome
Digest: sha256:cad108151c905d9f8aa41c58561c629314b6a958dde496ebd63f27f43191d794
Status: Image is up to date for sami1800/laravel-welcome:1.0
docker.io/sami1800/laravel-welcome:1.0
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$
```

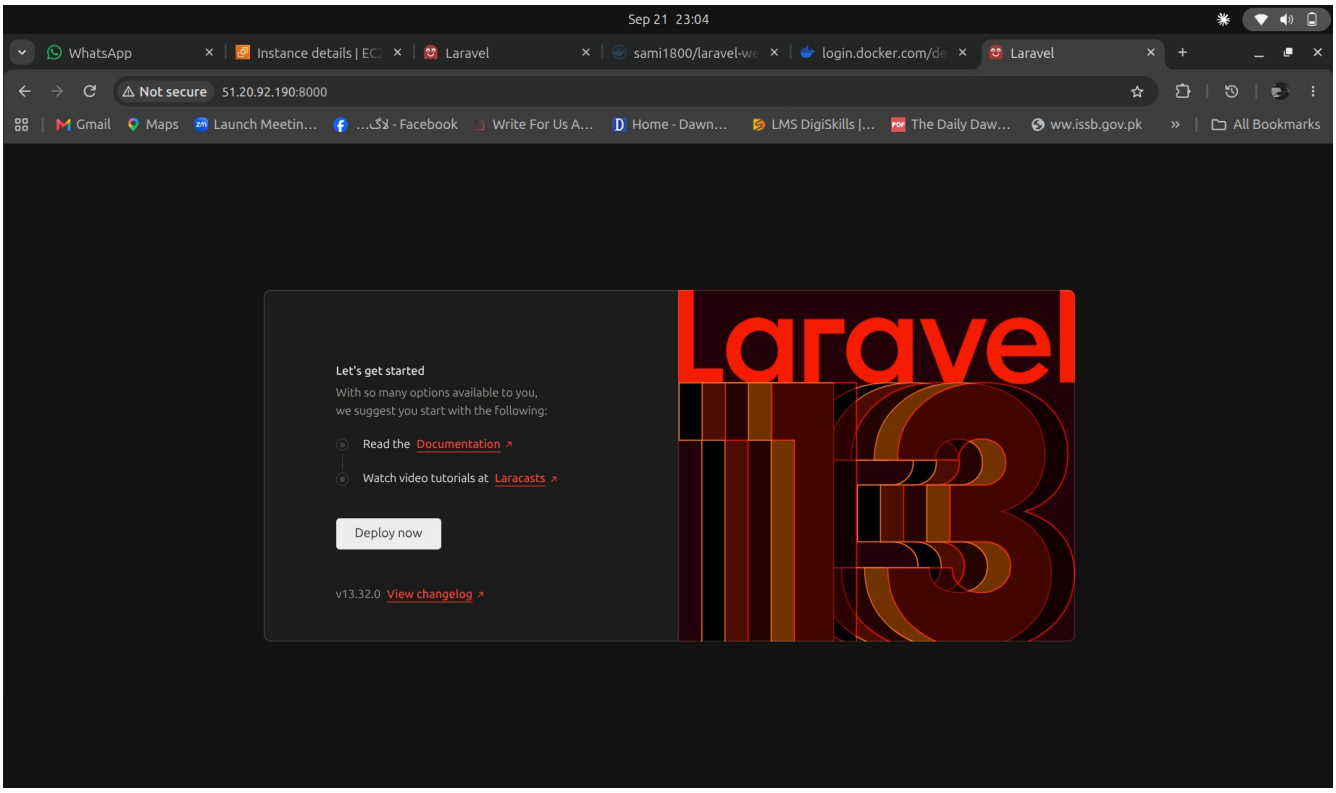
Running the pulled image on EC2 with **sudo docker run -d --name laravel-app -p 8000:8000 sami1800/laravel-welcome:1.0,** mapping the container's port 8000 to the host.

```
docker.io/sami1800/laravel-welcome:1.0
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker run -d \
  --name laravel-app \
  -p 8000:8000 \
  sami1800/laravel-welcome:1.0
0dd610df7b48cff1688993d49e2a22fc964e034dc29fa89f7a8b95b1855908e8
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$
```

Confirming with sudo docker ps that the **laravel-app container** is up and the port mapping **(0.0.0.0:8000->8000/tcp)** is active on the EC2 instance.

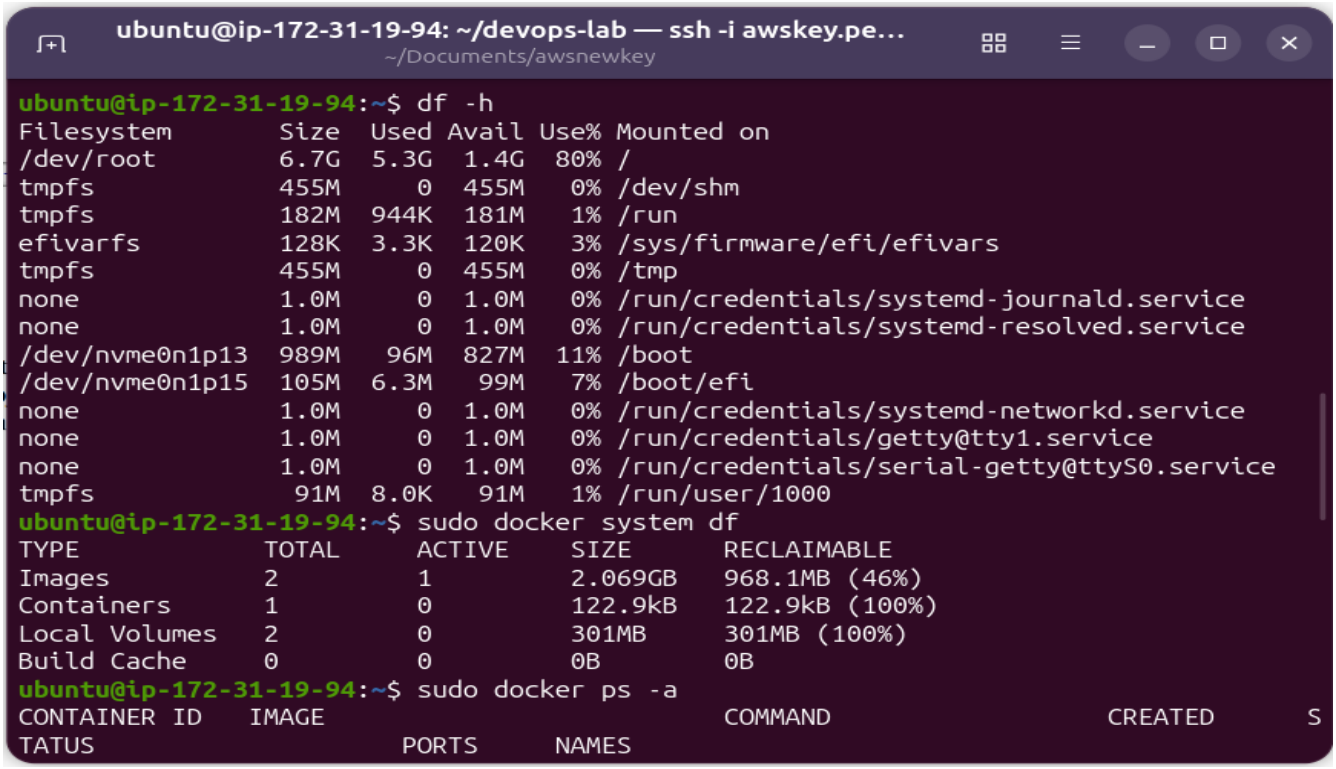
```
0dd610df7b48cff1688993d49e2a22fc964e034dc29fa89f7a8b95b1855908e8
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ sudo docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED
       STATUS              PORTS                                       NAMES
0dd610df7b48   sami1800/laravel-welcome:1.0   "docker-php-entrypoi…"   About a minute
 ago   Up About a minute   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp   laravel-app
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$
```

Now its running and **successful** and testing it finally through browser for confirmation. Opening the **AWS public IP on port 8000** in the browser. **The Laravel welcome page loads** successfully, confirming the full pipeline from Linux, **through Docker and Docker Hub,** to a live AWS EC2 deployment works end to end.



Push Changes to Pulled Docker Image from Docker Hub:

Initially, I have checked how much disk space is left on EC2 after creating image of Laravel default project with mysql, so space was low and I decided to delete the pushed container from EC2 disk space, as it is already on docker hub, when it is required later on I will pull back.



Now in this process we have removed the images from disk space.

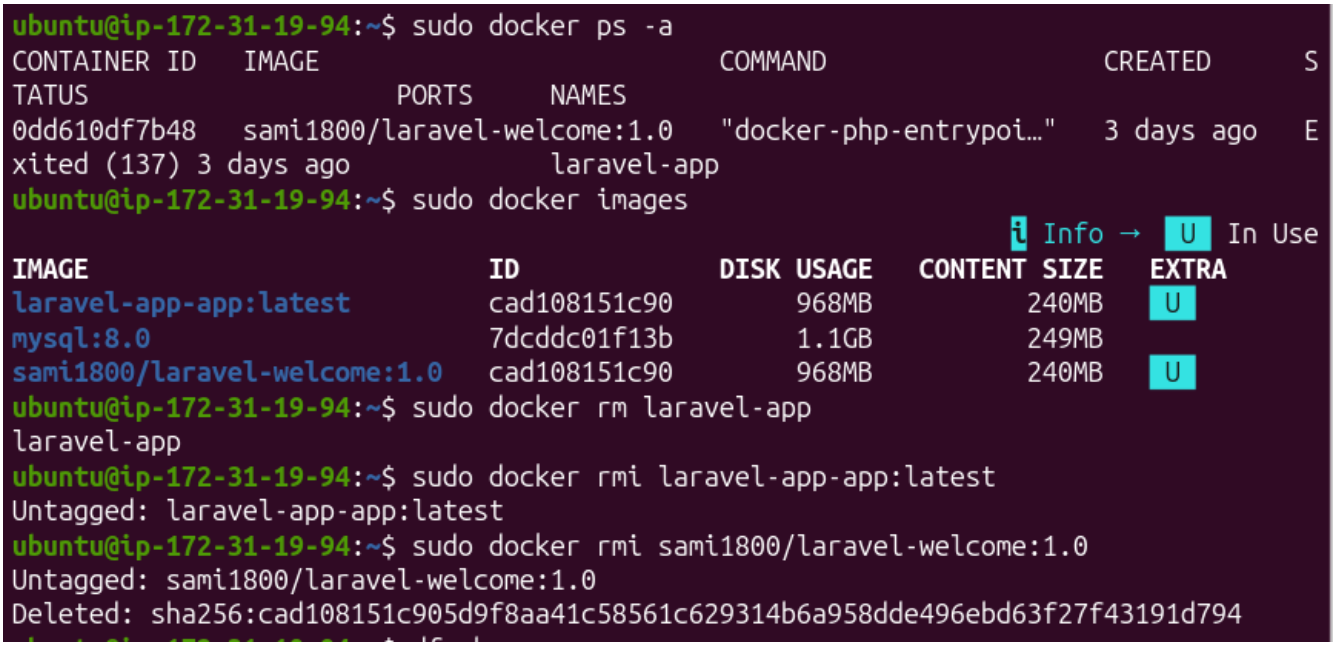


Image was pulled from docker hub.

```
ubuntu@ip-172-31-19-94:~/devops-lab/laravel-app$ cd ..
ubuntu@ip-172-31-19-94:~/devops-lab$ docker pull sami1800/laravel-welcome:1.0
1.0: Pulling from sami1800/laravel-welcome
d8f4c35935ec: Pull complete
cb3bed725cd6: Pull complete
1303a0d4ee60: Pull complete
866e8ec4e67d: Pull complete
4f4fb700ef54: Pull complete
81055ac1a39c: Pull complete
9b9c9ac6d652: Pull complete
aefd7737bbe4: Pull complete
6b37362b3da7: Pull complete
ea9b1dd160ca: Pull complete
99091603b490: Pull complete
96fc9acdcf33: Pull complete
f665ecff8fdb: Pull complete
0682a2b74e5f: Pull complete
d6aed5f5cedf: Pull complete
4383341a07a2: Download complete
Digest: sha256:cad108151c905d9f8aa41c58561c629314b6a958dde496ebd63f27f43191d794
Status: Downloaded newer image for sami1800/laravel-welcome:1.0
docker.io/sami1800/laravel-welcome:1.0
```

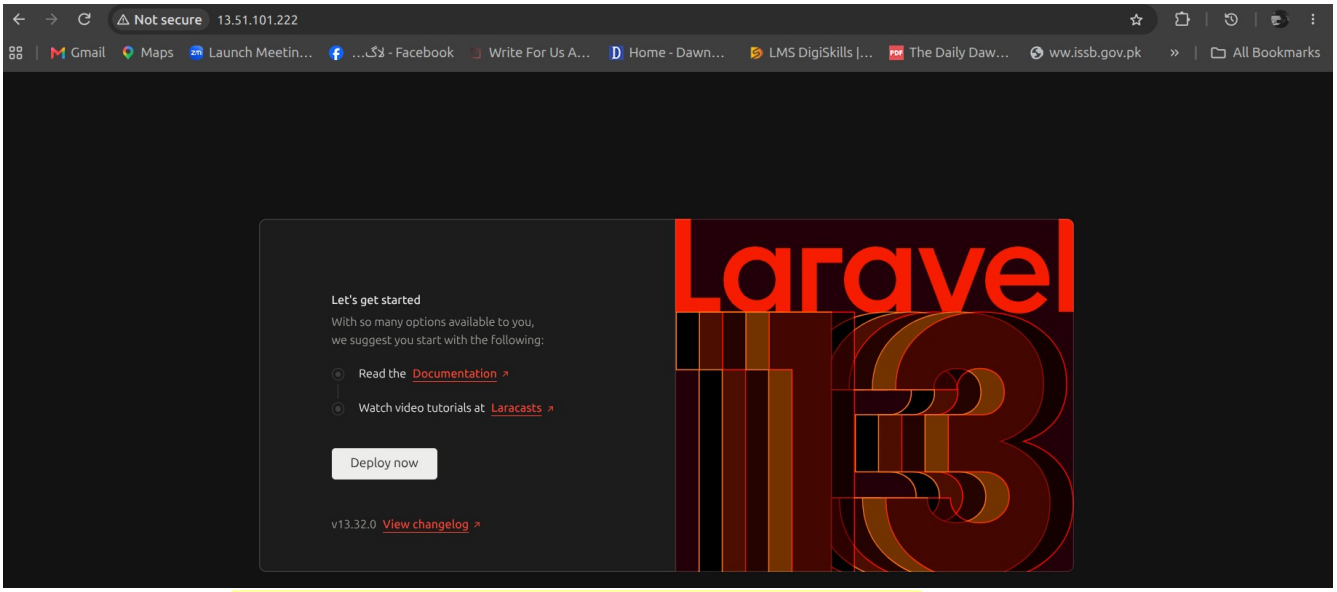
In this command instantiates and runs a Docker container named `laravel-student-app` in detached mode in the background using the image `sami1800/laravel-welcome:1.0`. It maps port `80` of the host EC2 instance to port `8000` inside the container, allowing incoming web traffic to access the Laravel application via HTTP.

```
ubuntu@ip-172-31-19-94:~/devops-lab$ docker run -d -p 80:8000 --name laravel-student-
app sami1800/laravel-welcome:1.0
d28ef0062bbc0f766c0093744f62a4d0d2b0c5a5adacba2fbcef27419c2b149b
```

I executed `docker ps` to verify that the `laravel-student-app` container was actively running and healthy on the EC2 instance. This was done to inspect its runtime status, container ID, and port bindings to ensure port 80 was properly mapped and ready to serve traffic.

```
ubuntu@ip-172-31-19-94:~/devops-lab$ docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED
  STATUS         PORTS                                     NAMES
d28ef0062bbc   sami1800/laravel-welcome:1.0   "docker-php-entrypoi…"   2 minutes ago
  Up 2 minutes   0.0.0.0:80->8000/tcp, [::]:80->8000/tcp   laravel-student-app
ubuntu@ip-172-31-19-94:~/devops-lab$
```

After this, http://13.51.101.222/ we have used public elastic ip to access it from browser to check whether latest pulled was accessible or not, and it was successful.



Now I executed `docker exec -it laravel-student-app bash` to open an interactive command shell directly inside the live container. This was done to bypass host limitations and operate directly within the containerized PHP environment. It gave me terminal access to run Laravel Artisan generators, edit files, and build the 3-tier architecture from inside.

```
ubuntu@ip-172-31-19-94:~/devops-lab$ docker exec -it laravel-student-app bash
root@d28ef0062bbc:/var/www# php artisan make:model Student -m

  INFO  Model [app/Models/Student.php] created successfully.

  INFO  Migration [database/migrations/2026_09_25_184700_create_students_table.php]
created successfully.

root@d28ef0062bbc:/var/www#
```

Now I wrote custom migration schema definitions into the generated migration file to establish table columns. This was done to define structural database attributes including `name`, `email` (unique constraint), and `course` fields. It ensured the database schema possessed the required structure before running the migration pipeline.

```
root@d28ef0062bbc:/var/www# cat << 'EOF' > database/migrations/$(ls database/migratio
ns t@d28ef0062bbc:/var/www# cat << 'EOF' > database/migrations/$(ls database/migratio
ns | grep create_students_table)
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
return new class extends Migration {
return new class extends Migration {
    public function up(): void {', function (Blueprint $table) {
        Schema::create('students', function (Blueprint $table) {
            $table->id();g('name');
            $table->string('name');->unique();
            $table->string('email')->unique();
            $table->string('course');
            $table->timestamps();
        });
    }
    public function down(): void {
    public function down(): void {ents');
```

Now I updated the `Student` Eloquent model class with `$fillable` array attributes for mass-assignment protection. This was done to allow safe programmatic insertion of `name`, `email`, and `course` fields coming from HTTP POST requests. It configured the object-relational mapping logic between the application controllers and the database tier.

```
    public function down(): void {
    public function down(): void {ents');
        Schema::dropIfExists('students');
    }
};F
EOF
root@d28ef0062bbc:/var/www# cat << 'EOF' > app/Models/Student.php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Student extends Model {
    use HasFactory;
    protected $fillable = ['name', 'email', 'course'];
}
EOF
root@d28ef0062bbc:/var/www#
```

Now I initialized an empty `database.sqlite` file and executed `php artisan migrate --force` inside the container. This was done to physically instantiate the Tier 3 persistence layer and execute all pending migration queries. It generated the underlying database tables directly inside the container without requiring user prompts.

```
root@d28ef0062bbc:/var/www# touch database/database.sqlite
php artisan migrate --force

  INFO  Preparing database.

  Creating migration table ..................................... 11.42ms DONE

  INFO  Running migrations.

  0001_01_01_000000_create_users_table ......................... 47.22ms DONE
  0001_01_01_000001_create_cache_table ......................... 30.42ms DONE
  0001_01_01_000002_create_jobs_table .......................... 46.29ms DONE
  2026_09_25_184700_create_students_table ...................... 15.00ms DONE

root@d28ef0062bbc:/var/www#
```

Now I created the `students.blade.php` template with Tailwind CSS, an input registration form, and a tabular display. This was done to implement Tier 1 (the presentation layer) so end-users could interact with the application through their web browsers. It provided the UI necessary to capture student details and dynamically display persisted database rows.

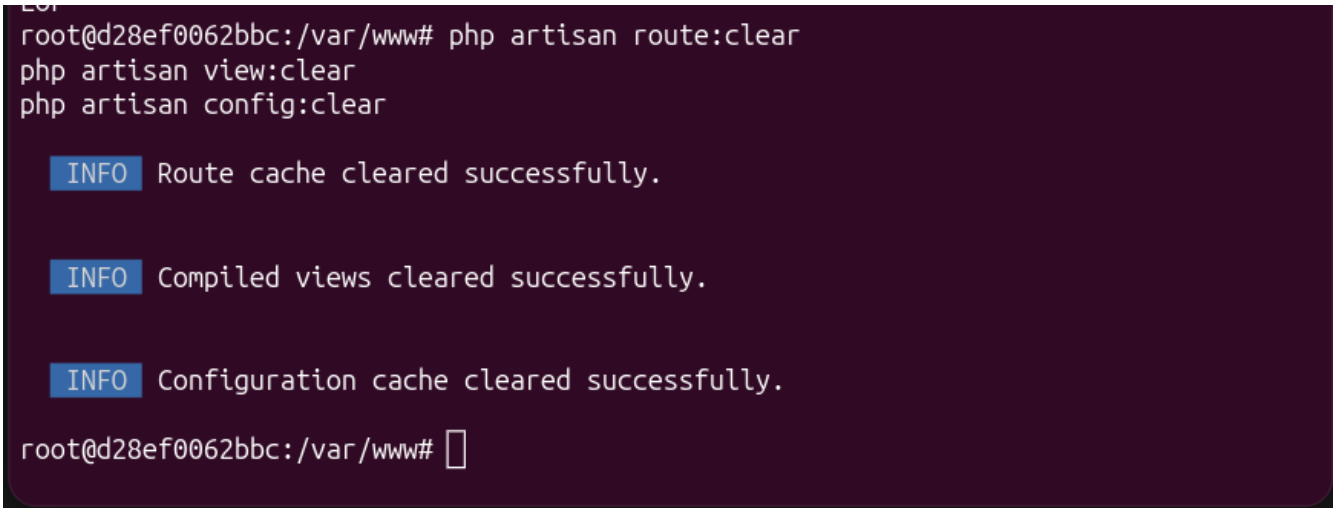
```
root@d28ef0062bbc:/var/www# cat << 'EOF' > resources/views/students.blade.php
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>3-Tier Laravel Portal</title>
    <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-slate-900 text-slate-100 min-h-screen p-8">
    <div class="max-w-4xl mx-auto">
        <header class="mb-8 border-b border-slate-700 pb-4">
            <h1 class="text-3xl font-bold text-sky-400">Student Portal (Tier 1 & Tier
 2)</h1>
            <p class="text-slate-400">Connected to Database Tier (Tier 3) on EC2</p>
        </header>

        @if(session('success'))
            <div class="mb-6 p-4 bg-emerald-900/60 border border-emerald-500 rounded
text-emerald-200">
                {{ session('success') }}
            </div>
```

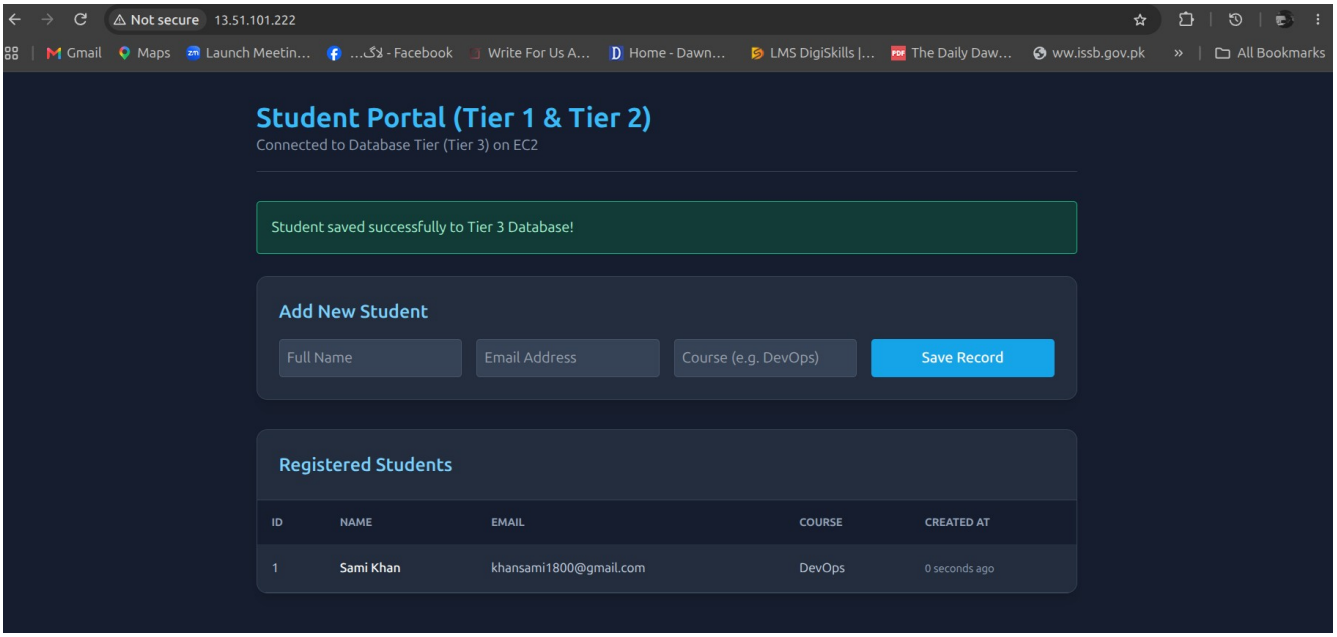
Now I updated `routes/web.php` with a `GET` route to fetch all student records and a `POST` route to validate and store incoming form data. This was done to bind Tier 1 user interactions with Tier 2 validation and Tier 3 database persistence. It enabled full round-trip data flow: capturing inputs, querying the SQLite database, and returning the updated view.

```
root@d28ef0062bbc:/var/www# cat << 'EOF' > routes/web.php
<?pt@d28ef0062bbc:/var/www# cat << 'EOF' > routes/web.php
<?php
use Illuminate\Support\Facades\Route;
use Illuminate\Support\Facades\Route;
use Illuminate\Http\Request;
use App\Models\Student;
Route::get('/', function () {
Route::get('/', function () {st()->get();
    $students = Student::latest()->get();ents'));
    return view('students', compact('students'));
});
Route::post('/students', function (Request $request) {
Route::post('/students', function (Request $request) {
    $validated = $request->validate([
        'name' => 'required',email|unique:students,email',
        'email' => 'required|email|unique:students,email',
        'course' => 'required'
    ]);
    Student::create($validated);
    Student::create($validated);success', 'Student saved successfully to Tier 3 Datab
    return redirect('/')->with('success', 'Student saved successfully to Tier 3 Datab
ase!');
});
```

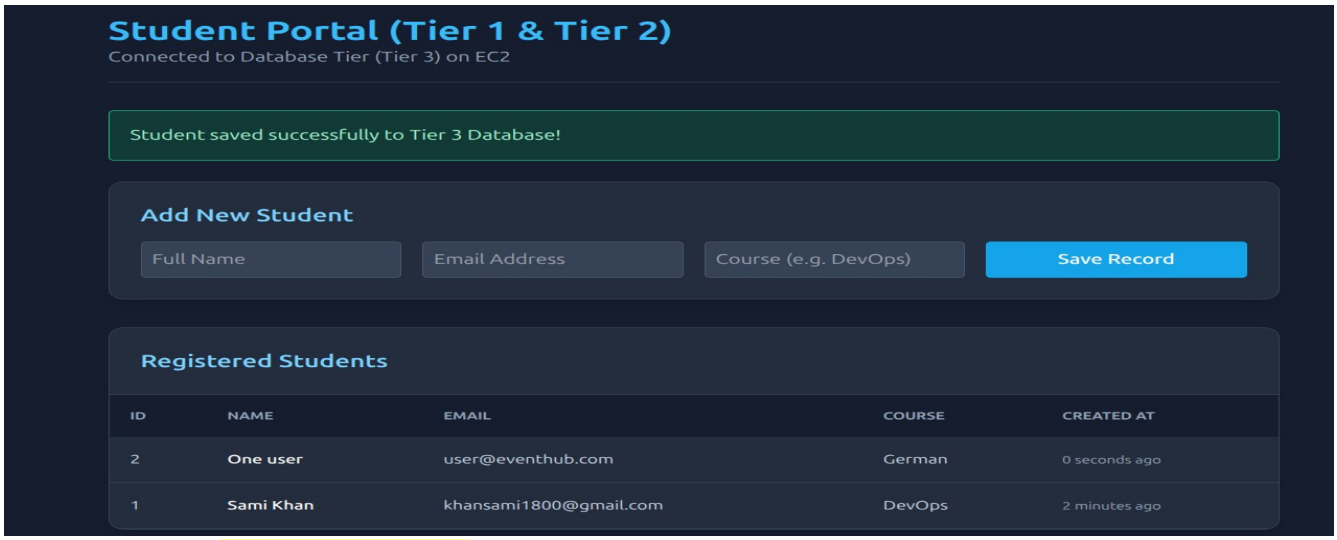
Now I executed cache-clearing commands for Laravel's routes, compiled Blade views, and runtime configurations.



Now I accessed the EC2 instance's public IP address in my web browser to verify that the containerized modifications rendered properly in the presentation tier. This was done to test real-world end-to-end functionality by submitting new student records through the interface and confirming they were dynamically saved to and retrieved from the underlying database tier.



Now I accessed the EC2 instance's public IP address in my web browser to verify that the containerized modifications rendered properly in the presentation tier. This was done to test real-world end-to-end functionality by submitting second student records through the interface and confirming they were dynamically saved to and retrieved from the underlying database tier.



Now I executed `docker commit` to snapshot the entire file system and modified state of the running `laravel-student-app` container into a new image tagged `2.0`. This was done to persist the new 3-tier database schema, code changes, and views into a standalone immutable Docker image. It prevented my modifications from being lost when the temporary container stops.

```
ubuntu@ip-172-31-19-94:~/devops-lab$ docker commit -m "Added student database tier an
d UI" laravel-student-app sami1800/laravel-welcome:2.0
sha256:09c006ace3e155d788902637ae03686524925a582c3a639fe8bbb134d732df32
ubuntu@ip-172-31-19-94:~/devops-lab$
```

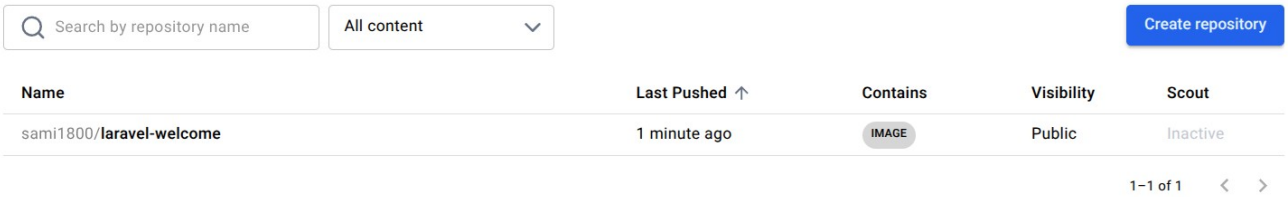
Now I ran `docker images | grep ...` to inspect the local Docker image repository on my EC2 instance. This was done to verify that version `2.0` had been compiled successfully alongside the original version `1.0`. It confirmed the new image ID and size were valid before initiating the network upload.

```
ubuntu@ip-172-31-19-94:~/devops-lab$ docker images | grep sami1800/laravel-welcome
WARNING: This output is designed for human readability. For machine-readable output,
please use --format.
sami1800/laravel-welcome:1.0   cad108151c90       968MB          240MB   U
sami1800/laravel-welcome:2.0   09c006ace3e1       968MB          240MB
ubuntu@ip-172-31-19-94:~/devops-lab$
```

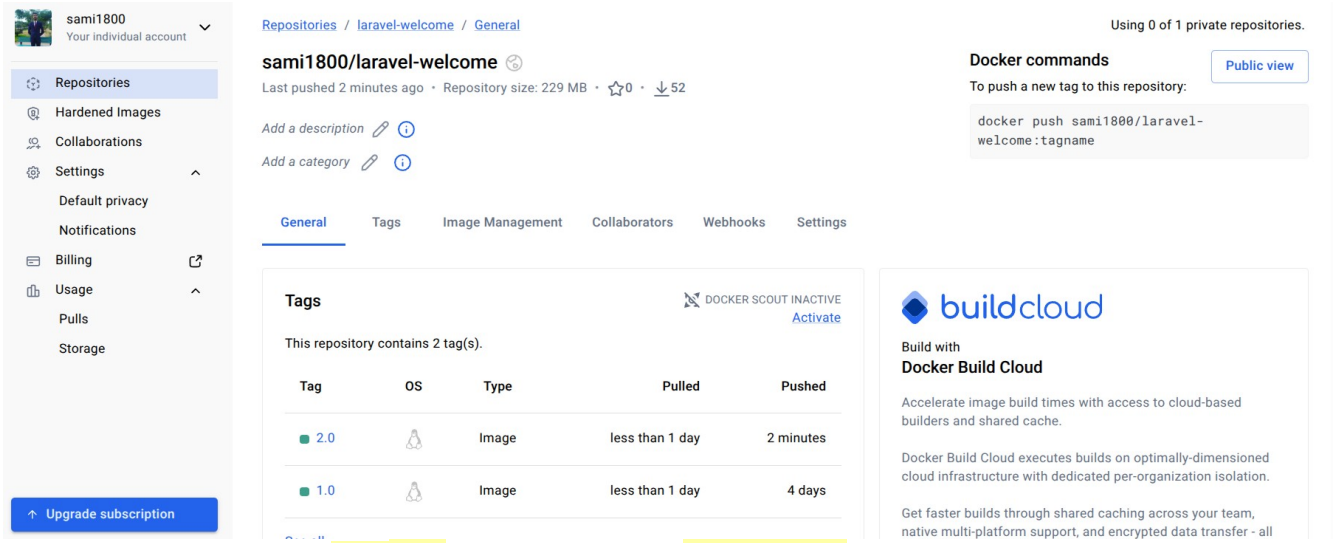
Now I executed `docker push sami1800/laravel-welcome:2.0` to publish the customized 3-tier image to my public Docker Hub repository. This was done to upload the newly created image layer (`fefc35599328`) while reusing previously cached base layers. It completed the DevOps lifecycle by making the updated version publicly retrievable and deployable on any server.

```
ubuntu@ip-172-31-19-94:~/devops-lab$ docker push sami1800/laravel-welcome:2.0
The push refers to repository [docker.io/sami1800/laravel-welcome]
9b9c9ac6d652: Layer already exists
99091603b490: Layer already exists
d6aed5f5cedf: Layer already exists
f665ecff8fdb: Layer already exists
4f4fb700ef54: Layer already exists
96fc9acdcf33: Layer already exists
d8f4c35935ec: Layer already exists
81055ac1a39c: Layer already exists
0682a2b74e5f: Layer already exists
ea9b1dd160ca: Layer already exists
cb3bed725cd6: Layer already exists
aefd7737bbe4: Layer already exists
6b37362b3da7: Layer already exists
1303a0d4ee60: Layer already exists
fefc35599328: Pushed
866e8ec4e67d: Layer already exists
2.0: digest: sha256:09c006ace3e155d788902637ae03686524925a582c3a639fe8bbb134d732df32
size: 3775
ubuntu@ip-172-31-19-94:~/devops-lab$
```

Now I navigated to the **Docker Hub web console** to verify that the remote repository registered the `2.0` tag update, indicated by the status showing the push completed "1 minute ago." This was done to validate the distribution phase of the deployment pipeline, ensuring that the newly packaged 3-tier artifact was successfully synced to the central registry and made available for downstream consumption.



Now I inspected the "Tags" tab on the Docker Hub console to review the complete version history of the repository, confirming that both the base version `1.0` and the newly updated `2.0` image are listed with their respective layer digests. This was done to verify tag immutability and confirm that our version-controlled artifact catalog accurately tracks all deployed application iterations..



Now I clicked into the **`2.0` tag** details view on Docker Hub to inspect the comprehensive layer metadata, compressed image size, and architecture specifications. This was done to verify that the newly committed image layer containing our database files and application code merged cleanly on top of the underlying parent image without bloat, confirming production readiness for deployment.

